UNIT 3 60 min

Boolean Logic and Truth Tables

Comparison operators, the logical operators and, or, not, precedence, and reading truth tables.

By the end of this unit you will be able to

- Write comparisons with `==`, `!=`, `<`, `>=` and explain the result
- Combine conditions with `and`, `or`, and `not`
- Build and read truth tables for a logic expression
- Explain short-circuit evaluation and why order matters
- Predict the result of an expression that mixes several operators

Comparisons produce booleans

A comparison asks a yes-or-no question and produces a `bool`. Every condition you will write in the next units is a comparison or a combination of them.

Operator	Meaning	Example	Result
`==`	equal to	`3 == 3`	`True`
`!=`	not equal to	`3 != 3`	`False`
`<` / `>`	less / greater than	`2 < 5`	`True`
`<=` / `>=`	less or equal / greater or equal	`5 >= 6`	`False`

> **Watch out**
> `=` assigns. `==` compares. Writing `if x = 5:` is a `SyntaxError`.

and, or, not

`and` is True only when **both** sides are True. `or` is True when **at least one** side is True. `not` flips the value.

logic.py

```
a = True
b = False
print(a and b)    # False
print(a or b)     # True
print(not a)      # False
print(5 > 3 and 2 == 2)   # True
```

Truth tables

A **truth table** lists every combination of inputs and the output for each. With two variables there are 2 × 2 = 4 rows. With three there are 8. Each extra variable doubles the rows. Use the table below to predict each column, then reveal the answers.

Truth tables for A and B INTERACTIVE

Predict each column first, then press **Reveal results**.

A	B	A and B	A or B	A != B (exclusive or)	not A
True	True	True	True	False	False
True	False	False	True	True	False
False	True	False	True	True	True
False	False	False	False	False	True

Note

The "exclusive or" column (called XOR) is True when exactly one input is True. It is the same as `!=` on two booleans.

Precedence

Operators run in a fixed order. From highest to lowest: comparisons, then `not`, then `and`, then `or`. Use parentheses whenever you are not sure. They make the code easier to read as well.

precedence.py

```
print(True or False and False)     # True: "and" runs first
print((True or False) and False)   # False
print(not True or True)            # True: "not" runs first
```

Short-circuit evaluation

Python stops evaluating as soon as the answer is known. In `x and y`, if `x` is False the whole expression is False and `y` is never run. In `x or y`, if `x` is True, `y` is skipped. This lets you write safe checks like `name is not None and name.strip() != ""`.

short_circuit.py

```
def check():
    print("check() was called")
    return True

print(False and check())   # check() never runs
print(True or check())    # check() never runs
```

Truthy and falsy values

Every value can be used as a condition. `0`, `0.0`, `""`, `[]`, and `None` are **falsy**. Everything else is **truthy**. Use this to write short checks like `if items:` to test for a non-empty list.

Predict the output THINK FIRST

Be careful with precedence and short-circuiting.

```
print(not 0)
```

Answer: `True`
0 is falsy, and `not` flips it.

```
print(3 == 3.0)
```

Answer: `True`
Int and float compare by value.

```
print("apple" < "banana")
```

Answer: `True`
Strings compare alphabetically, letter by letter.

```
print(1 < 2 < 3)
```

Answer: `True`
Python allows chained comparisons. It means `1 < 2 and 2 < 3`.

```
print(False and True or True)
```

Answer: `True`
`and` binds tighter, so it is `(False and True) or True`.

```
print(True or print("hi"))
```

Answer: `True`
The `or` already knows the answer, so `print("hi")` never runs.

Practice

▼ **Exercise 1 — Between 1 and 10**

Given `x = 7`, write one expression that is True only when `x` is from 1 to 10 inclusive. Print it.

▼ **Hint**

Python lets you chain comparisons. You can also write it with `and`.

▼ **Exercise 2 — NAND table**

Write a truth table for `not (A and B)`. Predict every row before you run the code, then check.

Discussion

1. Restaurants say "you may have soup or salad." Is that an inclusive or an exclusive or? How would you write each in Python?
2. Why is `a or b` safe to write when `a` is a check like `user is None`?
3. Can you write an expression equivalent to `A or B` using only `and` and `not`? Try it with a truth table.

Key terms

Boolean

A value that is either `True` or `False`.

Comparison operator

An operator that compares two values and returns a boolean: `==`, `!=`, `<`, `>`, `<=`, `>=`.

Logical operator

`and`, `or`, and `not`, which combine or invert boolean values.

Truth table

A table listing every combination of inputs and the resulting output of a logic expression.

XOR (exclusive or)

True when exactly one of two inputs is True. In Python, `a != b` on booleans.

Operator precedence

The fixed order in which operators are evaluated. Parentheses override it.

Short-circuit evaluation

Stopping a boolean expression as soon as its result is known, so later parts are never evaluated.

Truthy / falsy

Values that count as True or False in a condition. Falsy values include `0`, `""`, `[]`, and `None`.

Chained comparison

A comparison such as `1 < x < 10`, which Python evaluates as two comparisons joined by `and`.

See all terms in the glossary →